AutoGrader: Implementation Plan

Rule-driven evaluator for C programs

Member	Name	Roll No.
1		
2		
3		
4		

1. Idea

We will write a Python program that takes a student's C file and the teacher's rules file, runs the code safely, checks it on every metric, and gives one final grade with feedback. Same code and same rules should always give the same grade.

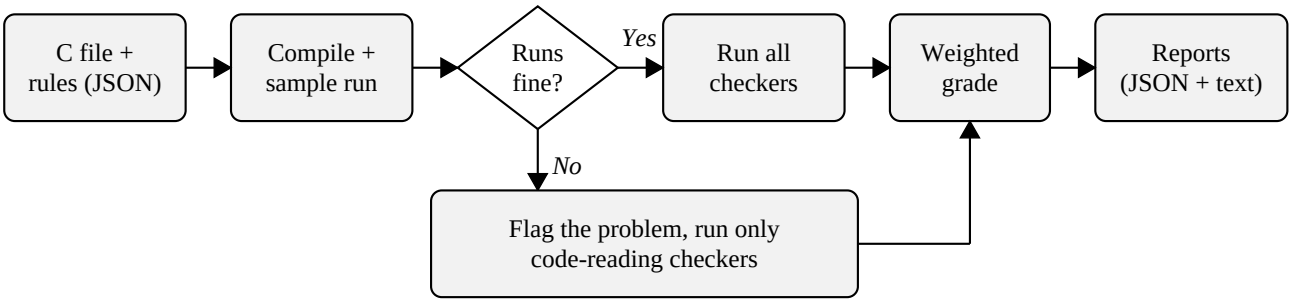

Figure 1: Overall flow of the grader

2. Key design choices

- **Rules in a JSON file.** Limits, metrics, weights and thresholds live outside the code. Adding a metric means adding one checker and one line in the JSON.
- **Safe running.** Each run happens in a temp folder with CPU, memory and time limits. Crashes and infinite loops are noted in the report instead of stopping the grader.
- **Same output for every checker:** a score from 0 to 1, the evidence, and one line of feedback.
- **Final grade** = 100 x sum of (weight x score).

3. Checkers

Metric	How we will check it
Correctness	Run the hidden test cases and compare outputs.
Syntax	Compile with the given gcc flags and count warnings.
Structure, modularity	lizard and cppcheck for function count, length and complexity.
Edge cases	Tricky inputs (0, negatives, INT_MAX, extra spaces, empty input) and random ones, compared with the teacher's correct solution (Figure 2).
Complexity	Time the program on growing input sizes and see how fast the time grows.
Memory safety	ASan, UBSan and Valgrind for leaks and bad memory access.
Readability	Local LLM (Ollama) at temperature 0 with a fixed seed. Answers are saved and reused.
Originality	Compare code with other submissions and with LLM/online solutions after renaming variables. High similarity is flagged for the teacher.

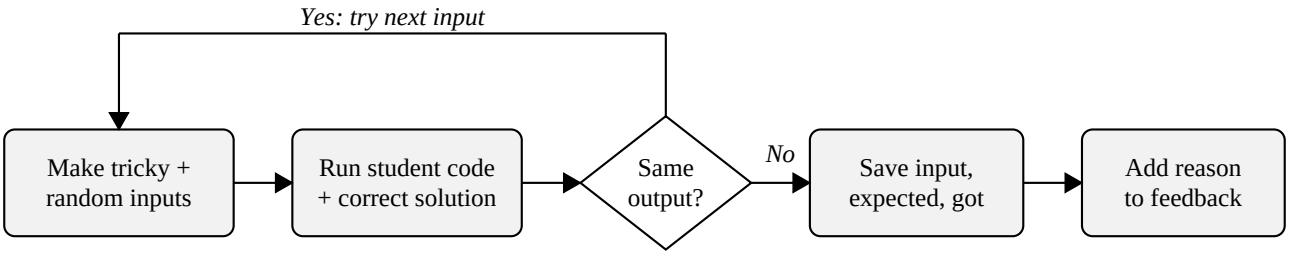

Figure 2: How failing test cases are found

4. Making the grade repeatable

- Fixed seeds for random tests, and saved LLM answers.
- Complexity is judged by the growth pattern (median of 5 runs), not raw timings.
- Each report stores the rules file hash and tool versions.

5. Work split

Member	Work
1	Main controller, rules loading, safe running, final grade
2	Test runner, edge-case generator, complexity check
3	Compiler, structure and memory-safety checks
4	LLM review, similarity check, reports, README, video

6. Progress so far

- Read the assignment and the sample *add.c* example.
- Finalised the design, rules format and tools.

7. Timeline

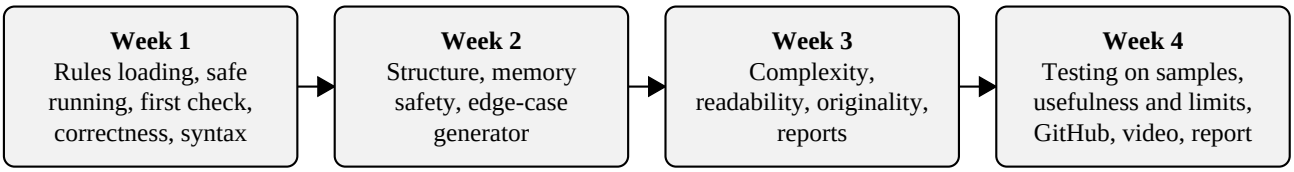

Figure 3: Weekly plan